

Multiplicity

Place multiplicity notations near the ends of an association. These symbols indicate the number of instances of one class linked to one instance of the other class. For example, one company will have one or more employees, but each employee works for one company only.

Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class. Multiplicity **constrains the number** of related objects.

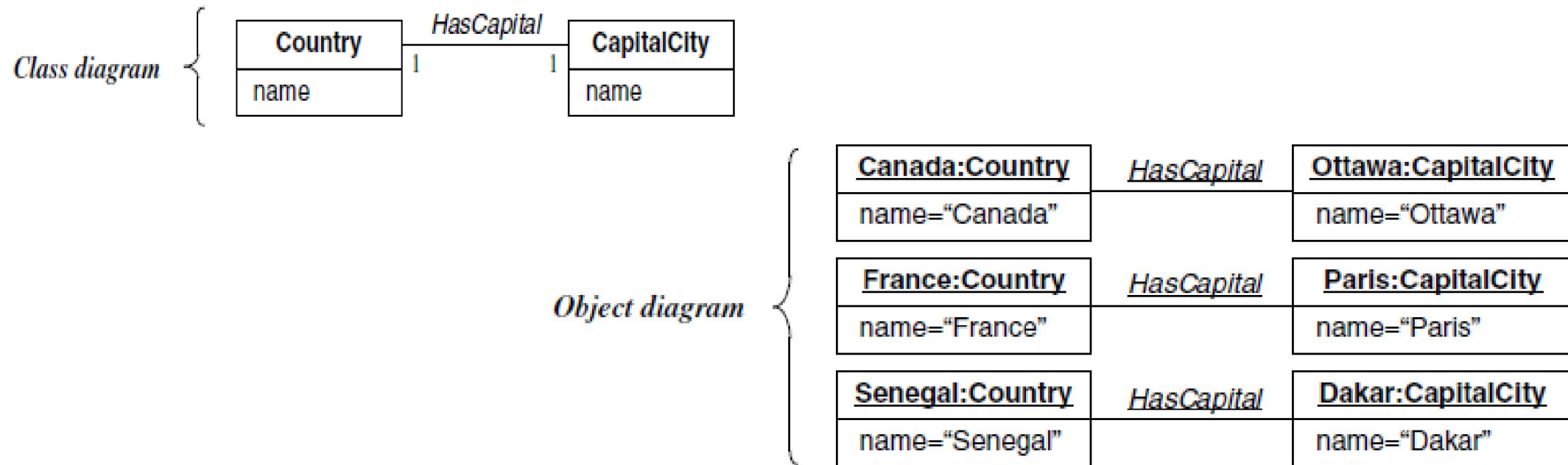


Figure 3.8 One-to-one association. Multiplicity specifies the number of instances

The table below shows the interval values that can be used.

Possible values	Meaning
1..1	Exactly one object. This is the default multiplicity of an association according to UML.
0..*	No object or at least one.
1..*	At least one object.
$m..m$	Exactly m objects. Example: 0..0 is no object, 3..3 is exactly three objects. In the diagram, an exact number of objects is displayed as a single number. Example: the interval 2..2 is displayed as 2 in the diagram.
$m..n$	At least m objects but no more than n objects. Example: 3..8 is at least three and at the most 8 objects. 0..1 is no object or one object.

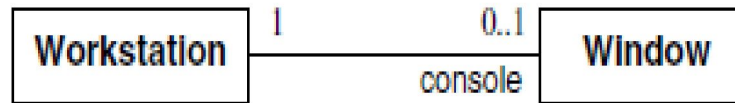


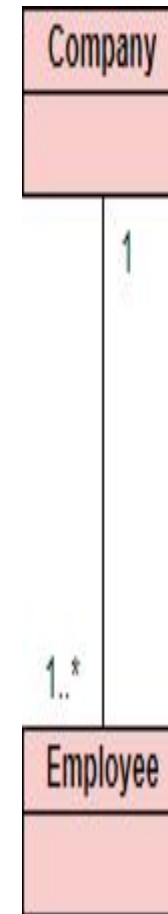
Figure 3.9 Zero-or-one multiplicity. It may be optional whether an object is involved in an association.

illustrates zero-or-one multiplicity. A workstation may have one of its windows designated as the console to receive general error messages. It is possible, however, that no console window exists.

Multiplicity is a definition of **cardinality** - i.e. **number of elements** - of some collection of elements by providing an inclusive interval of non-negative integers to specify the allowable number of instances of described element. Multiplicity interval has some **lower bound** and (possibly infinite) **upper bound**:

multiplicity-range ::= [lower-bound '..'] upper-bound
lower-bound ::= natural-value-specification
upper-bound ::= natural-value-specification | '*'

For example, one company will have one or more employees, but each employee works for one company only.



Multiplicities examples:

1 Exactly one, no more and no less

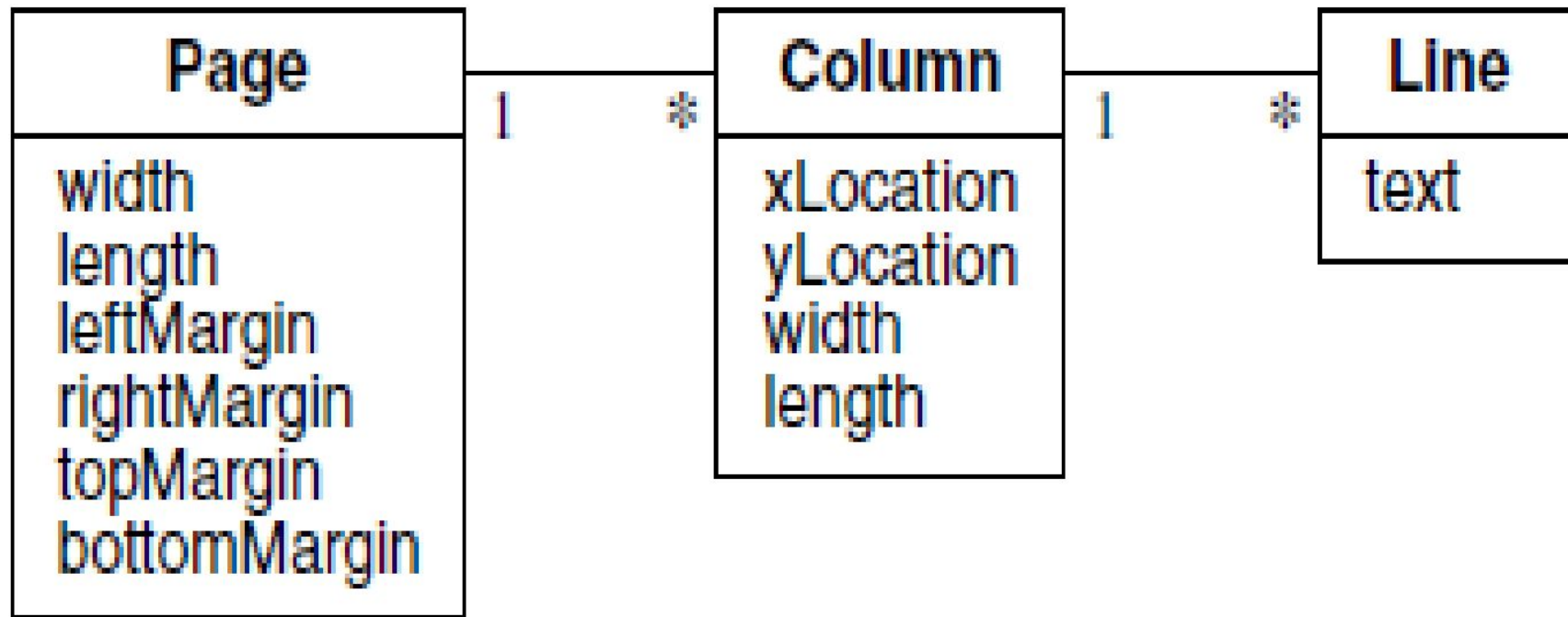
0..1 Zero or one

* Many

0..* Zero or many

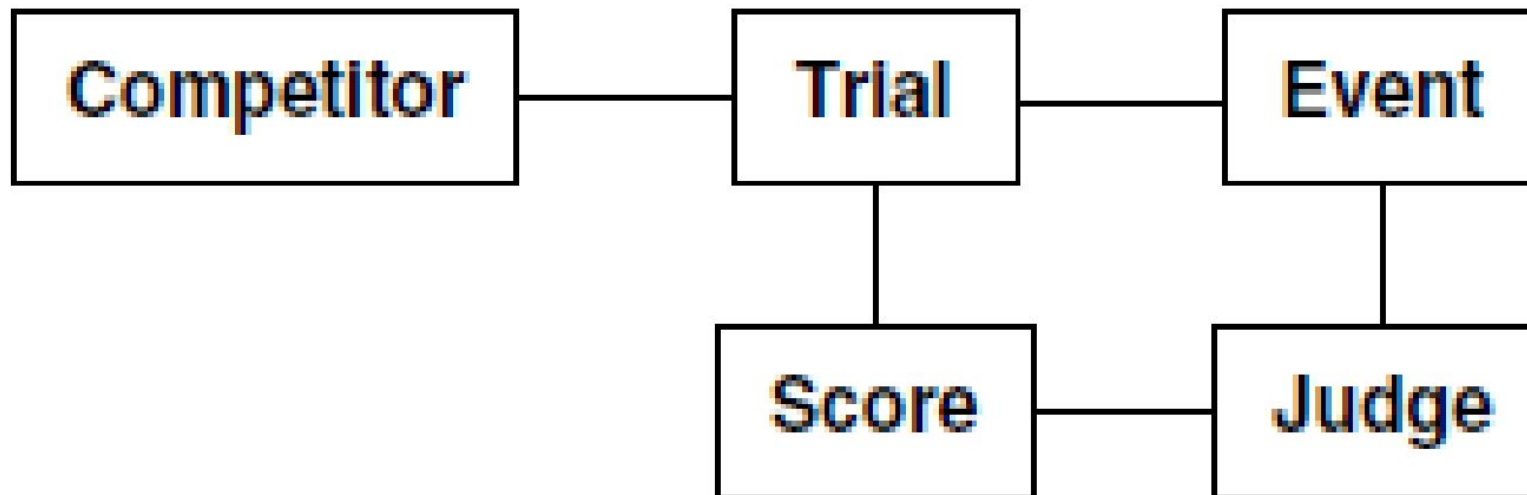
1..* One or many

Exercise – Design a class diagram for a computer system for laying out a newspaper. The system handles newspaper pages which may contain, among other things, columns of text. The user may edit the width and length of a column of text, move it around on a page, or move it from one page to another. note a column is displayed on exactly one page.



Draw a class diagram that might be used in developing a system to simplify the scheduling and scoring of judged athletic competitions such as gymnastics, diving, and figure skating. There are multiple events and competitors. Each competitor may enter several events and each event has many competitors. Each event has several judges who subjectively rate the performance of competitors in that event. A judge rates every competitor for an event. In some cases, a judge may score more than one event. Trials are the focus of the competition. Each trial is an attempt by one competitor to perform his or her best in one event. A trial is scored by the panel of judges for that event and a net score determined.

Add multiplicity to the diagram.



Prepare a class model to describe undirected graphs. An undirected graph consists of a set of vertices and a set of edges. Edges connect pairs of vertices. Your model should capture only the structure of graphs (i.e., connectivity) and need not be concerned with layout such as location of vertices or lengths of edges.

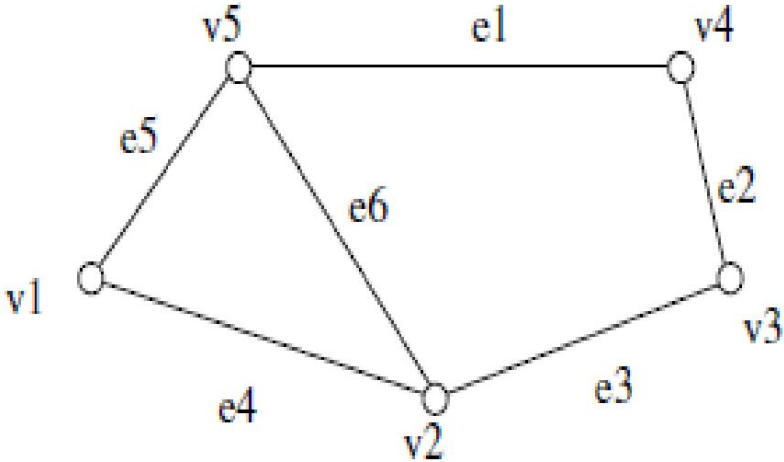
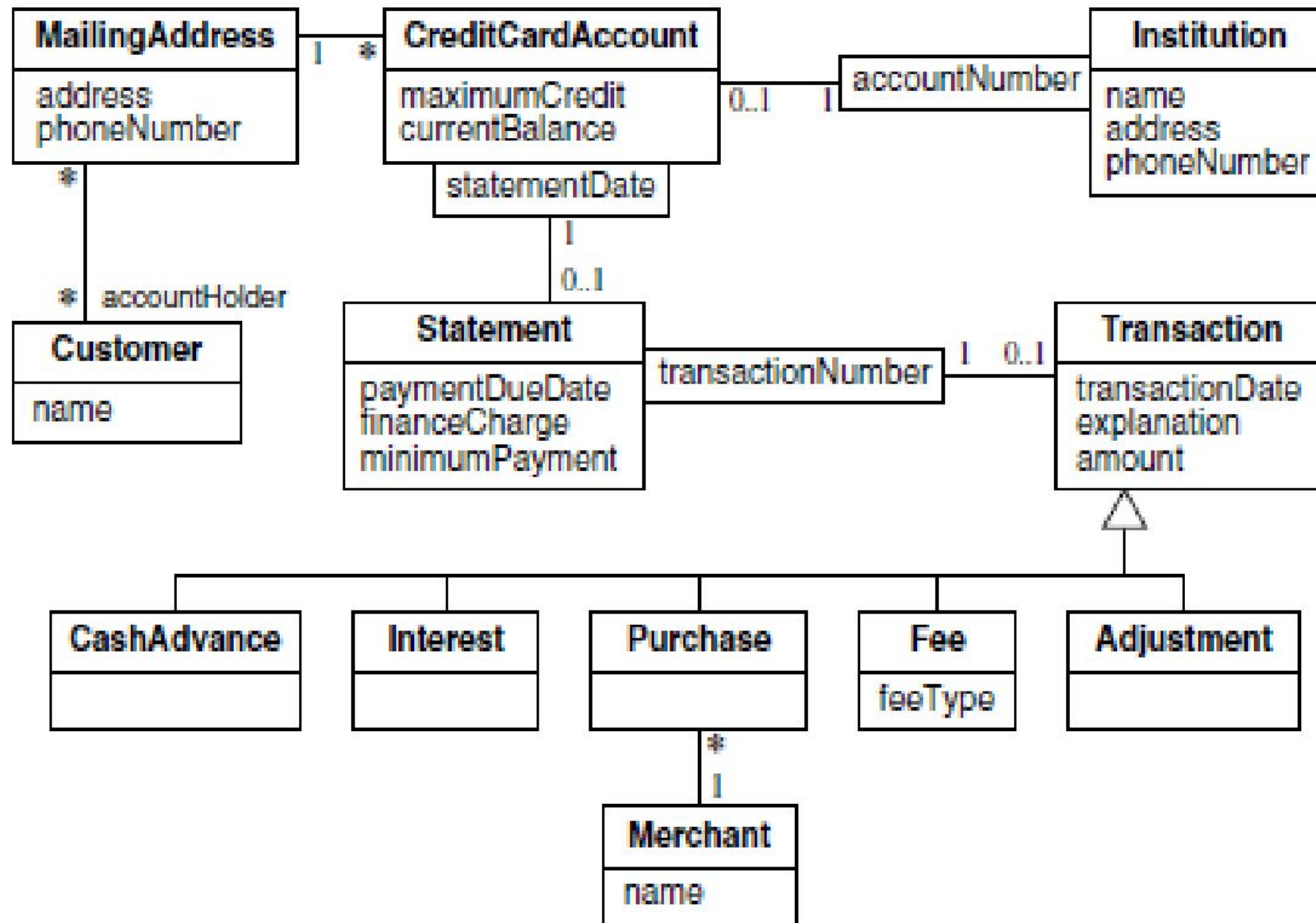


Figure E3.10 Sample undirected graph

Class model for managing credit card accounts



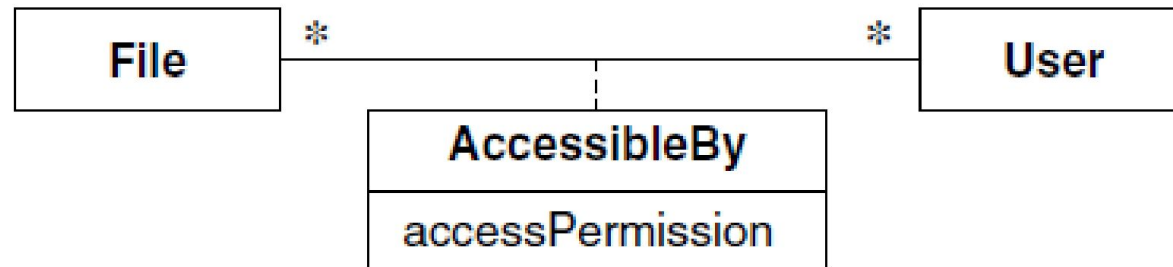
3.28 (6) Prepare a class diagram for the dining philosopher problem. There are 5 philosophers and 5 forks around a circular table. Each philosopher has access to 2 forks, one on either side. Each fork is shared by 2 philosophers. Each fork may be either on the table or in use by one philosopher. A philosopher must have 2 forks to eat.

Association Classes

An **association class** is an association that is also a class. Like the links of an association, the instances of an association class derive identity from instances of the constituent classes.

Like a class, an association class can have attributes and operations and participate in associations.

You can find association classes by looking for adverbs in a problem statement or by abstracting known values



/etc/termcap
/etc/termcap
/usr/doe/.login

read
read-write
read-write

John Doe
Mary Brown
John Doe

Many-to-many associations provide a compelling rationale for association classes. Attributes for such associations unmistakably belong to the link and cannot be ascribed to either object.

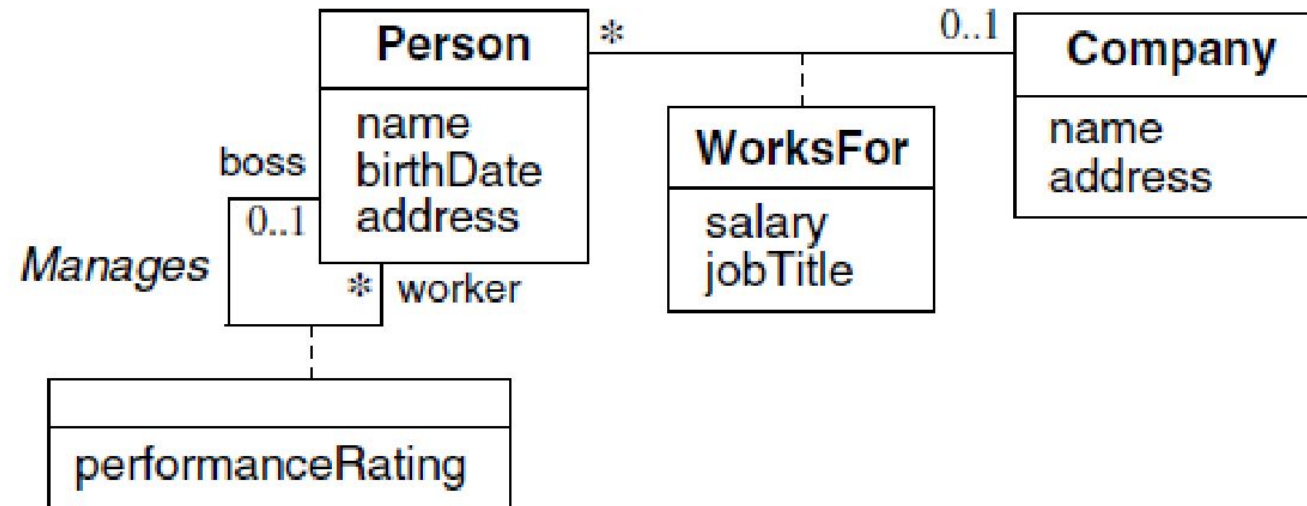


Figure 3.18 Association classes. Attributes may also occur for one-to-many and one-to-one associations.

Figure 3.18 presents attributes for two one-to-many associations. Each person working for a company receives a salary and has a job title. The boss evaluates the performance of each worker. Attributes may also occur for one-to-one associations.

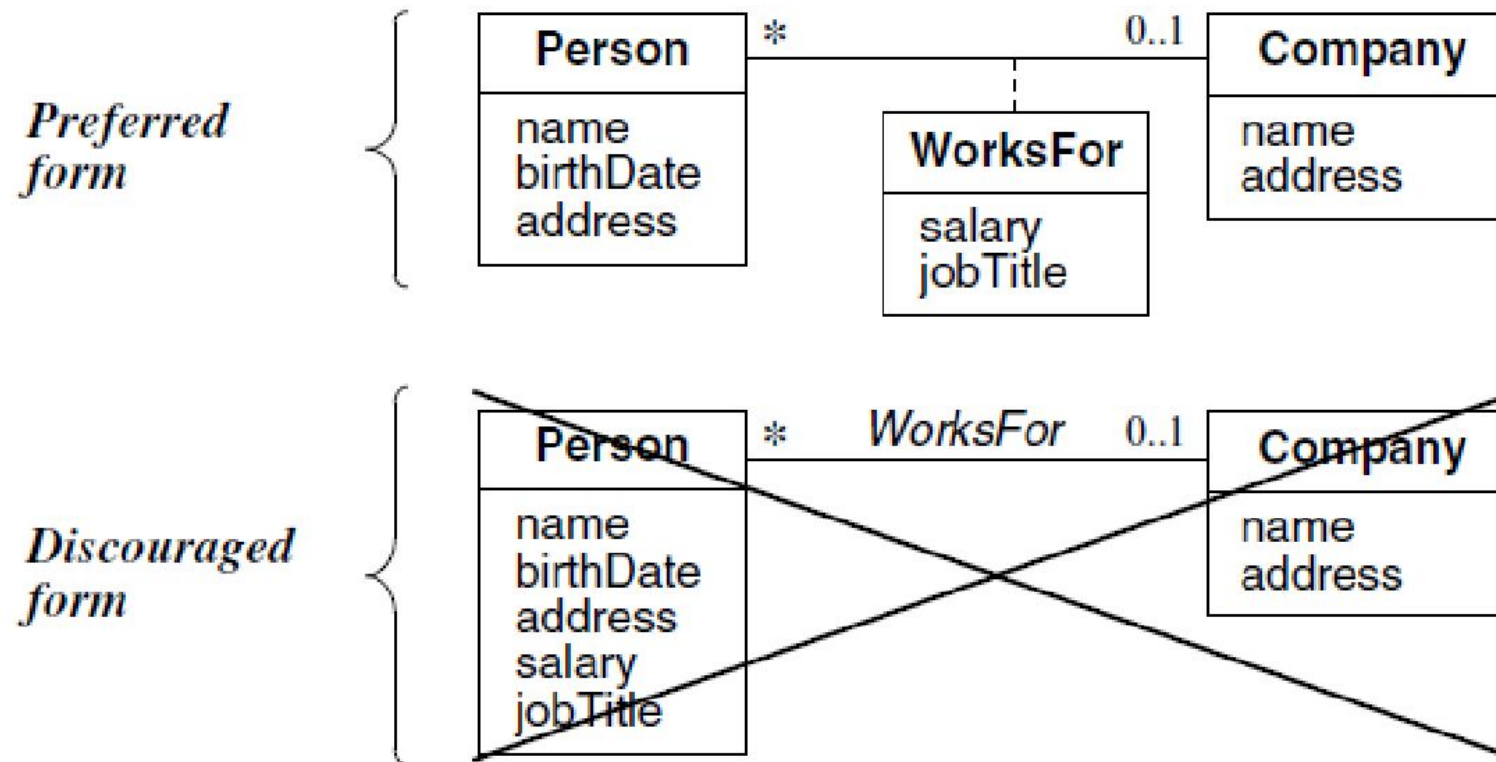


Figure 3.19 Proper use of association classes. Do not fold attributes of an association into a class.

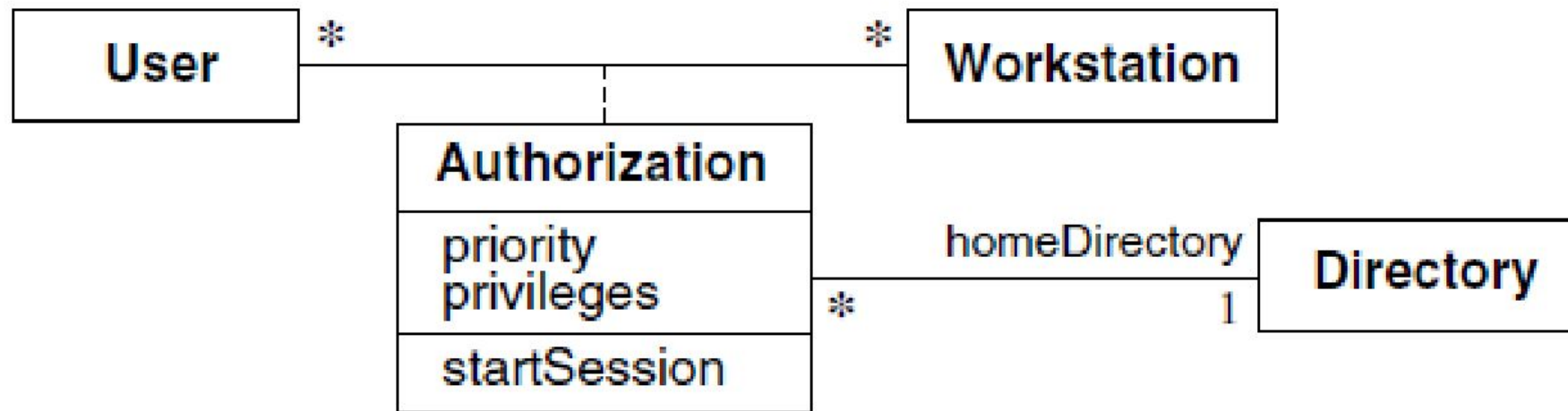


Figure 3.20 An association class participating in an association. Association classes let you specify identity and navigation paths precisely.

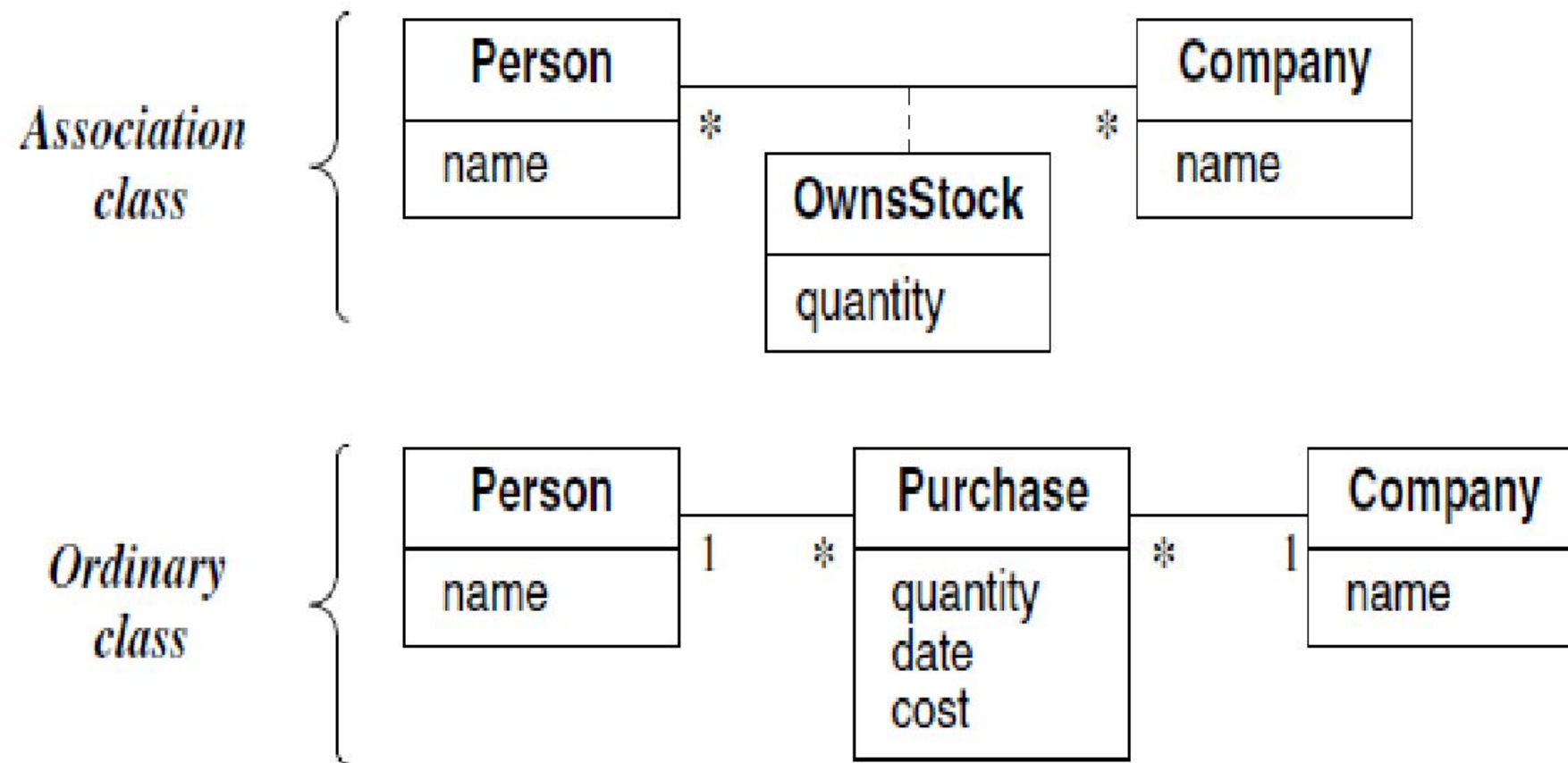


Figure 3.21 Association class vs. ordinary class. An association class is much different than an ordinary class.

Qualified Associations

A **qualified association** is an association in which an attribute called the **qualifier** disambiguates the objects for a “many” association end.

It is possible to define qualifiers for one-to-many and many-to-many associations. A qualifier selects among the target objects, reducing the effective multiplicity, from “many” to “one.”

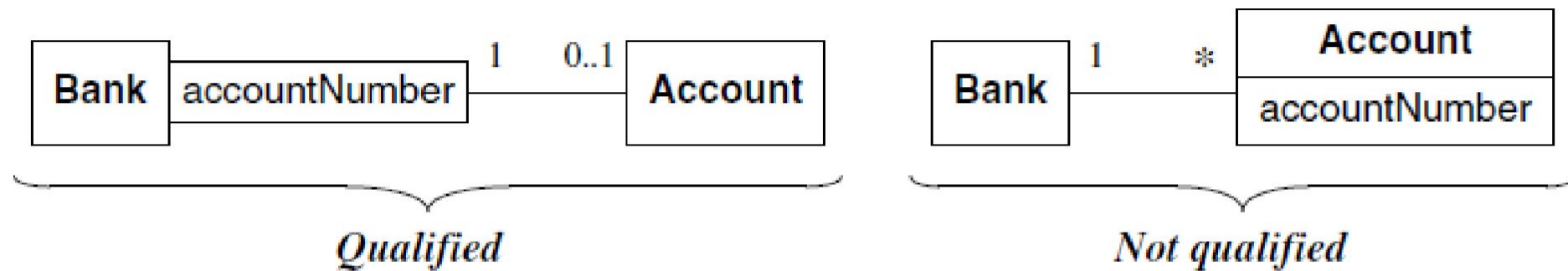


Figure 3.22 Qualified association. Qualification increases the precision of a model.

Both models are acceptable, but the qualified model adds information. The qualified model adds a multiplicity constraint, that the combination of a bank and an account number yields at most one account. The qualified model conveys the significance of account number in traversing the model, as methods will reflect. You first find the bank and then specify the account number to find the account.

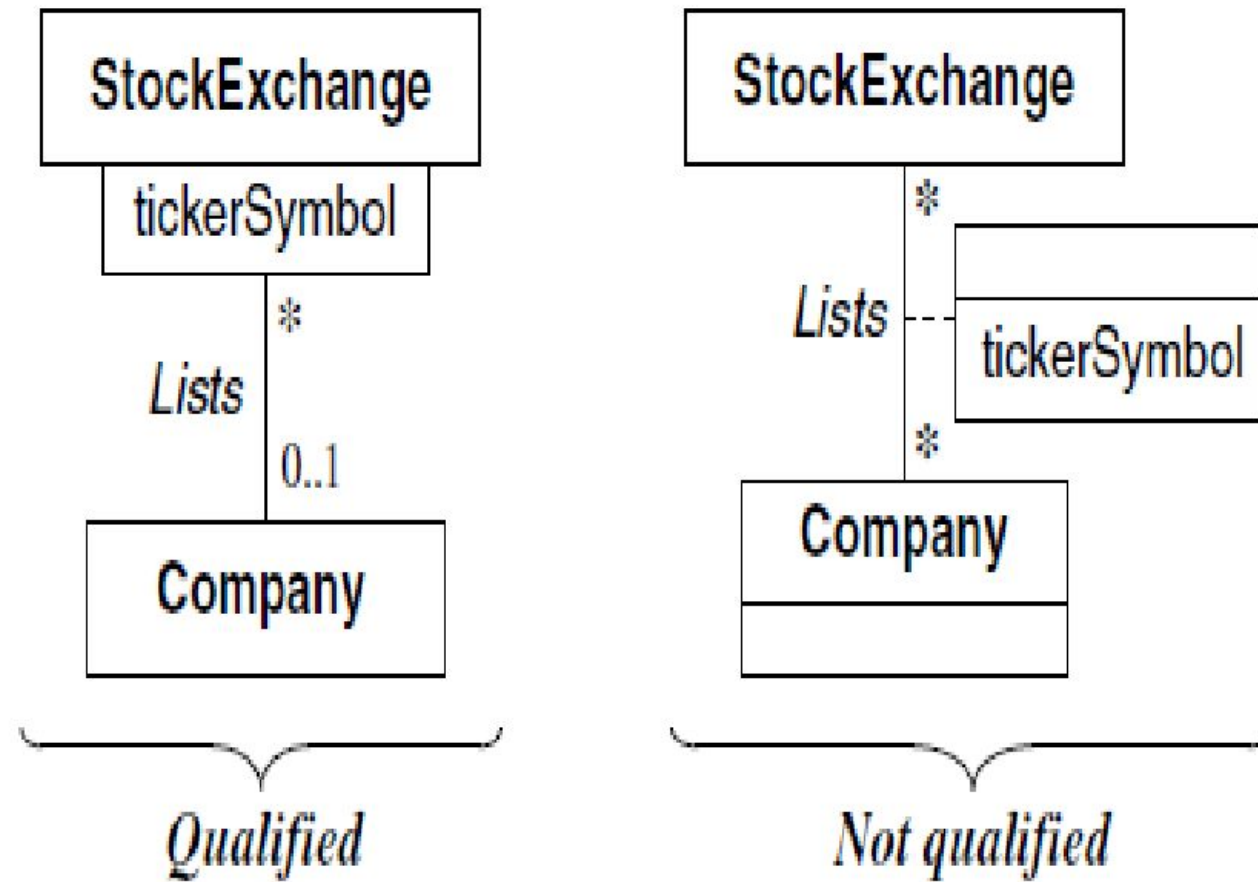
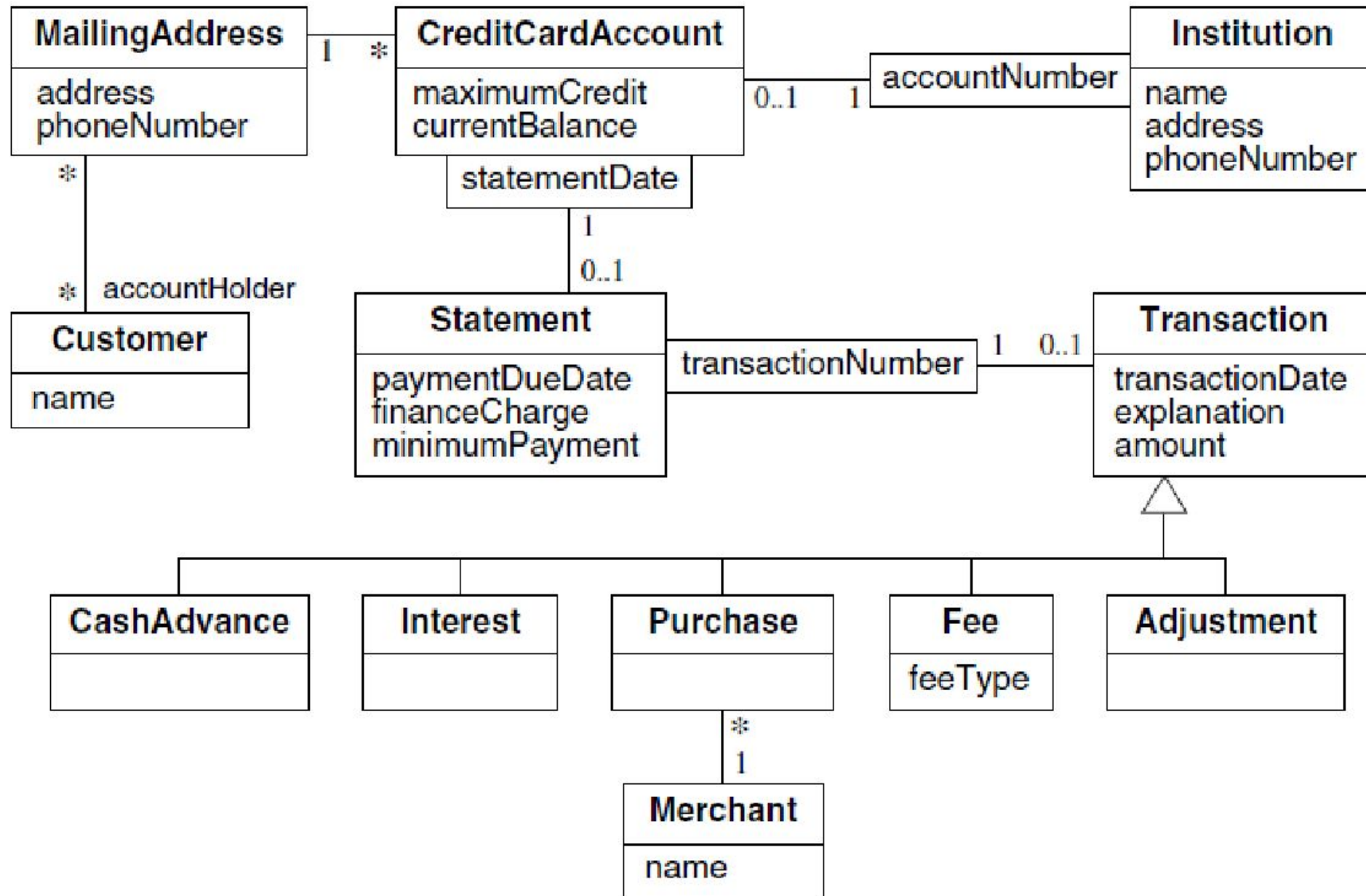


Figure 3.23 Qualified association. Qualification also facilitates traversal of class models.



Prepare a class diagram for
A person may have multiple magazine subscriptions. A magazine has multiple subscribers. For each subscription, it is important to track the date and amount of each payment as well as the current expiration date.

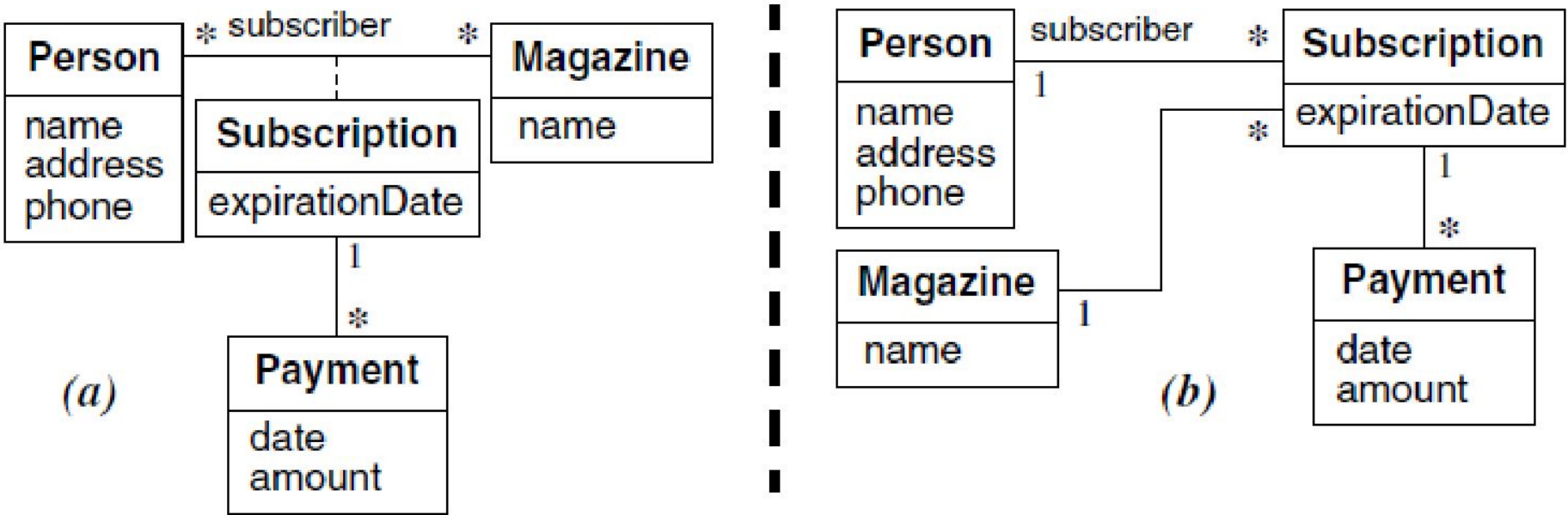


Figure E3.13 Class diagram for magazine subscriptions

Modeling

Enumerations- A data type is a description of values. Data types include numbers, strings, and enumerations. An **enumeration** is a data type that has a finite set of values.

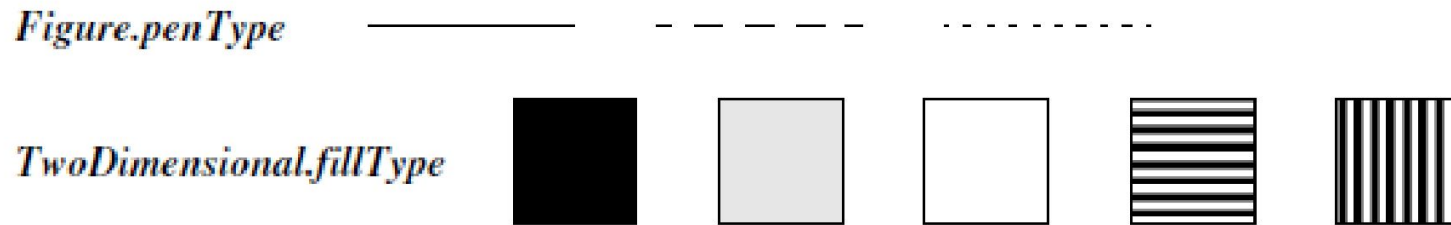
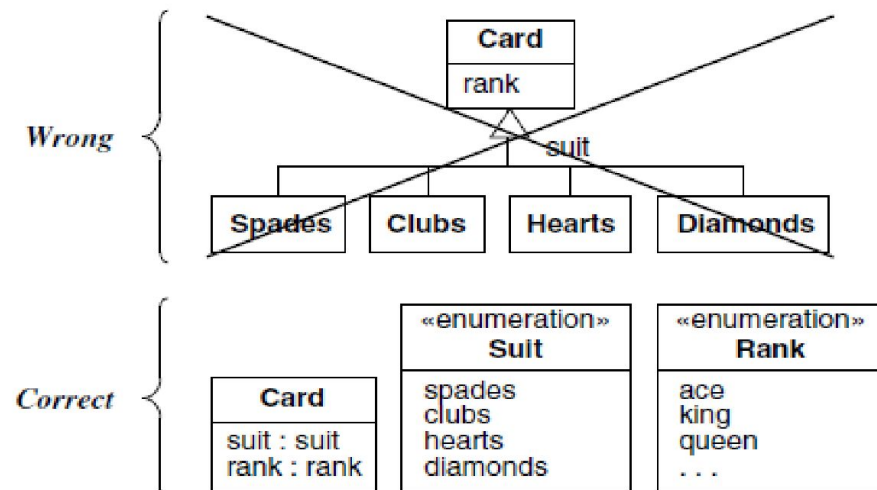


Figure 4.1 Examples of enumerations. Enumerations often occur and are important to users. Implementations must enforce the finite set of values.

Note:-

Do not use a generalization to capture the values of an enumerated attribute. An enumeration is merely a list of values; generalization is a means for structuring the description of objects.



Multiplicit

Multiplicity is a constraint on the cardinality of a set.

Multiplicity for an attribute specifies the number of possible values for each instantiation of an attribute. The most common specifications are a mandatory single value [1], an optional single value [0..1], and many [*].

Note:-

Multiplicity specifies whether an attribute is mandatory or optional (in database terminology whether an attribute can be null).

Person
name : string [1] address : string [1..*] phoneNumber : string [*] birthDate : date [1]

Figure 4.3 Multiplicity for attributes. You can specify whether an attribute is single or multivalued, mandatory or optional.

Scope

The *scope* indicates if a feature applies to an object or a class. An underline distinguishes features with class scope (static) from those with object scope.

Visibilit

V*isibility* refers to the ability of a method to reference a feature from another class and has the possible values of *public*, *protected*, *private*, and *package*.

Visibility allows to constrain the usage of a **named element**, either in **namespaces** or in access to the element.

UsedWith **classes**, **packages**, **generalizations**, **element import**, **package import**.

UML has the following types of **visibility**:

- public
- package
- protected
- private

Any method can freely access *public* features.

Only methods of the containing class and its descendants via inheritance can access *protected* features. (Protected features also have package accessibility in Java.)

Only methods of the containing class can access *private* features.

Methods of classes defined in the same package as the target class can access *package* features.

Sr. No	The character	precedes	Role
1	“+”	public features	Any method can freely access public features.
2	“#”	protected features	Only methods of the containing class and its descendants via inheritance can access protected features. (Protected features also have package accessibility in Java.)
3	“-”	private features	Only methods of the containing class can access private features.
4	“~”	package features	Methods of classes defined in the same package as the target class can access package features.

Navigability. Conceptually, an association may be traversed in either direction. However, an implementation may support only one direction. The UML shows navigability with an arrowhead on the association end attached to the target class. Arrowheads may be attached to zero, one, or both ends of an association.

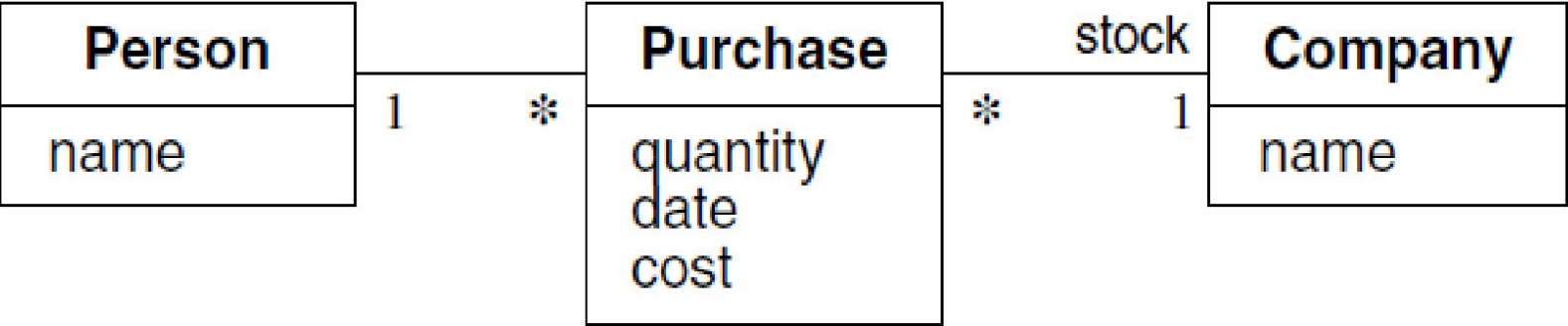
N-ary

Associations

By default associations between two classes is binary associations

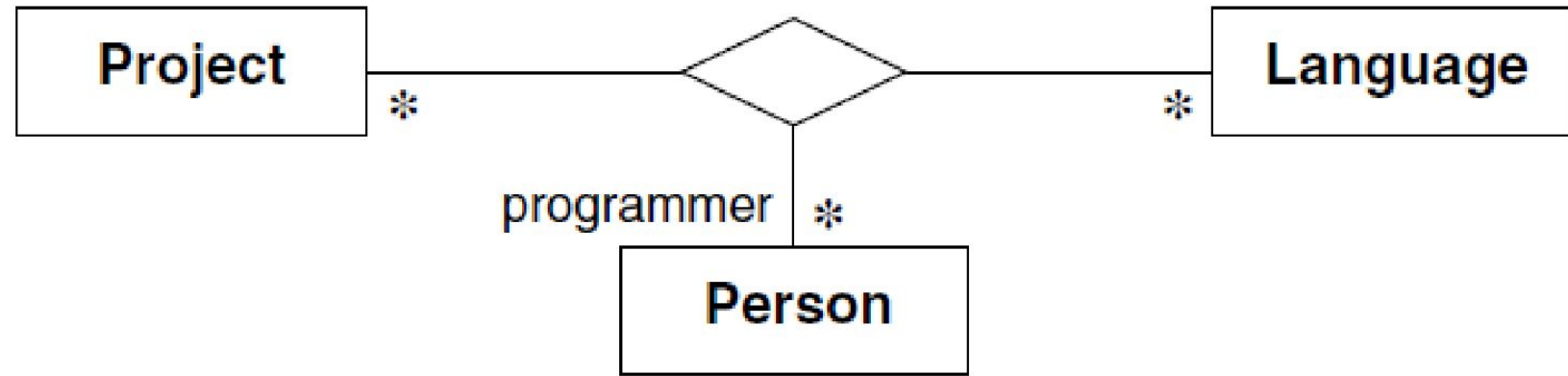
A nonatomic n-ary association—a person makes the purchase of stock in a company...

Can be restated as...

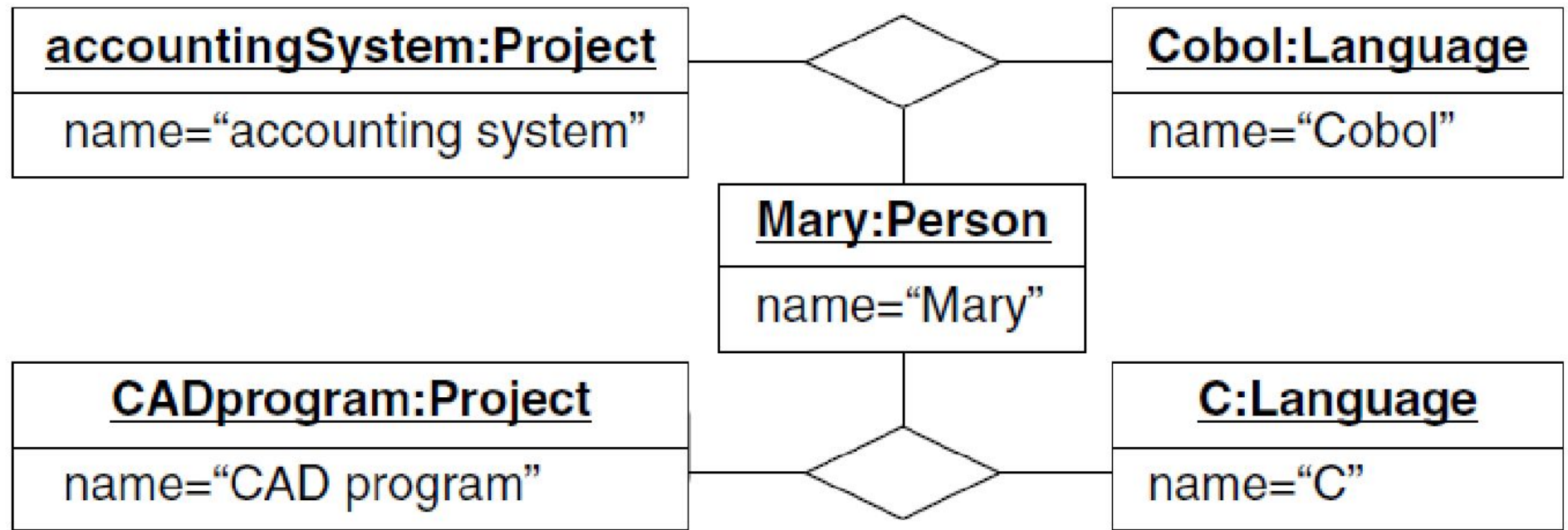


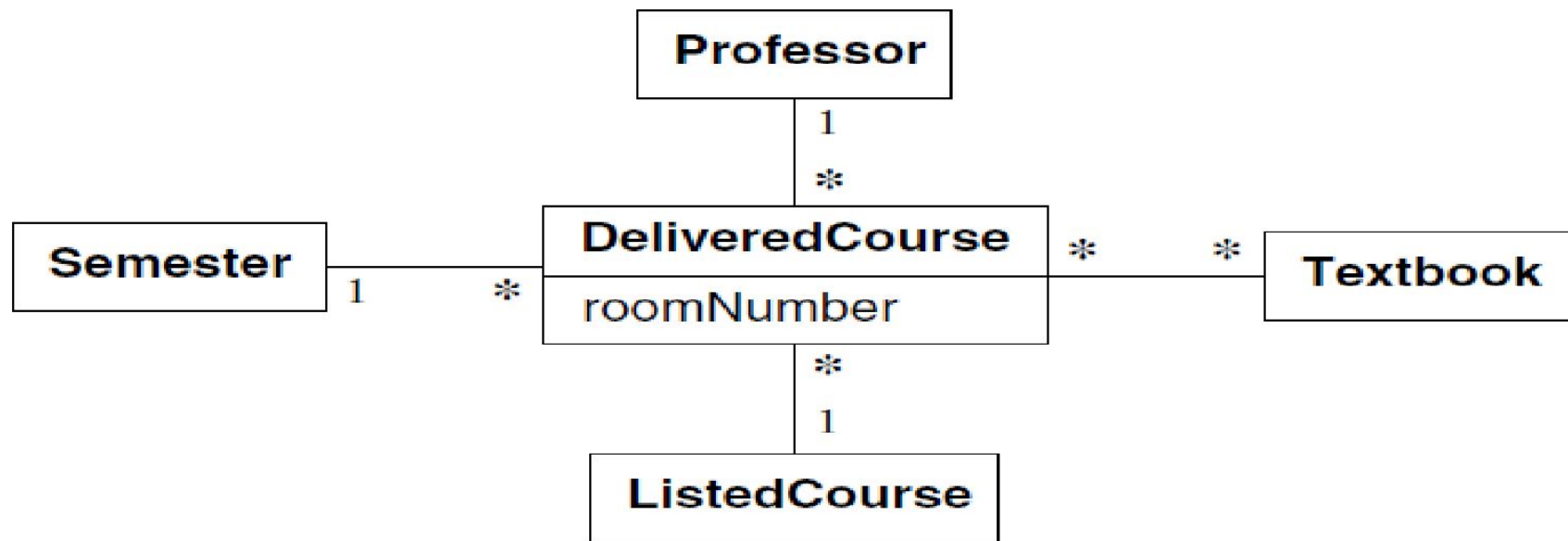
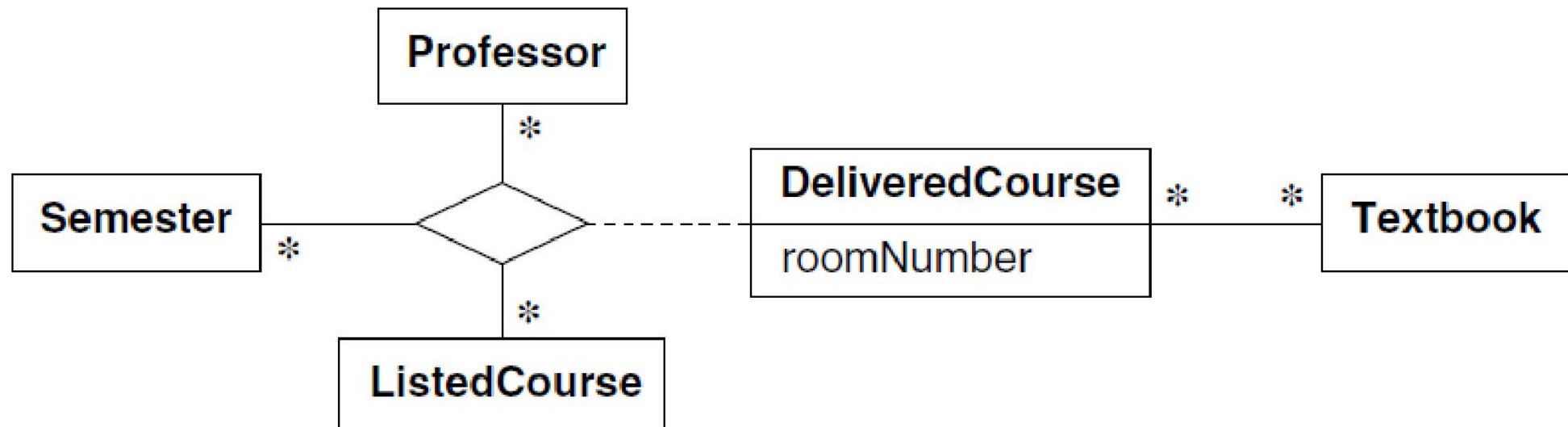
You should try to avoid n-ary associations—most of them can be decomposed into binary associations,

*Class
diagram*



*Instance
diagram*





Aggregatio

Aggregation is a strong form of association in which an aggregate object is *made of* constituent parts

An assembly with many kinds of constituent parts corresponds to many aggregations. For example, a *LawnMower* consists of a *Blade*, an *Engine*, many *Wheels*, and a *Deck*. *LawnMower* is the assembly and the other parts are constituents

LawnMower to *Blade* is one aggregation, *LawnMower* to *Engine* is another aggregation, and so on.

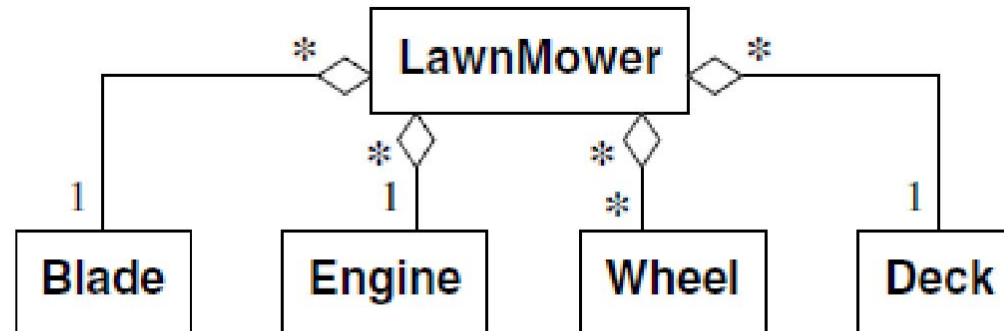


Figure 4.9 Aggregation. Aggregation is a kind of association in which an aggregate object is made of constituent parts.

property of aggregation

The most significant property of aggregation is *transitivity*—that is, if A is part of B and B is part of C , then A is part of C .

Aggregation is also *antisymmetric*—that is, if A is part of B , then B is not part of A .

Aggregation Versus Association

Aggregation is a special form of association, ~~not an independent~~ concept

If two objects are tightly bound by a part-whole relationship, it is an aggregation.

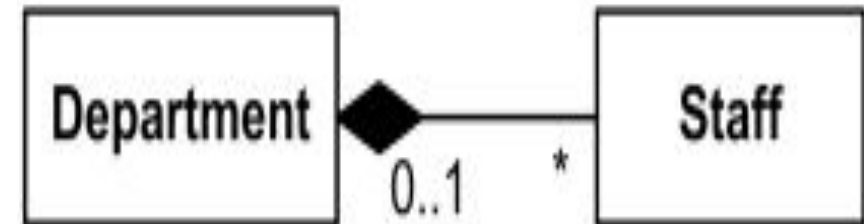
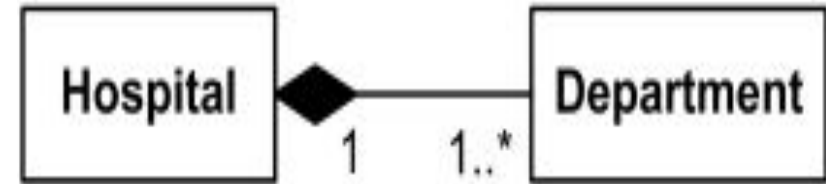
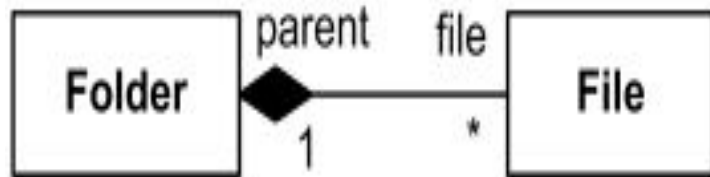
If the two objects are usually considered as independent, even though they may often be linked, it is an association.

Composite aggregation (composition) is a "strong" form of **aggregation** with the following characteristics:

- ❖ it is binary association,
- ❖ it is a *whole/part* relationship,
- ❖ a part could be included in *at most one* composite (whole) at a time, and
- ❖ if a composite (whole) is deleted, all of its composite parts are "normally" deleted with it.

Notation

Composite aggregation is depicted as a binary association decorated with a **filled black diamond** at the aggregate (whole) end.



Aggregation Versus

Composition

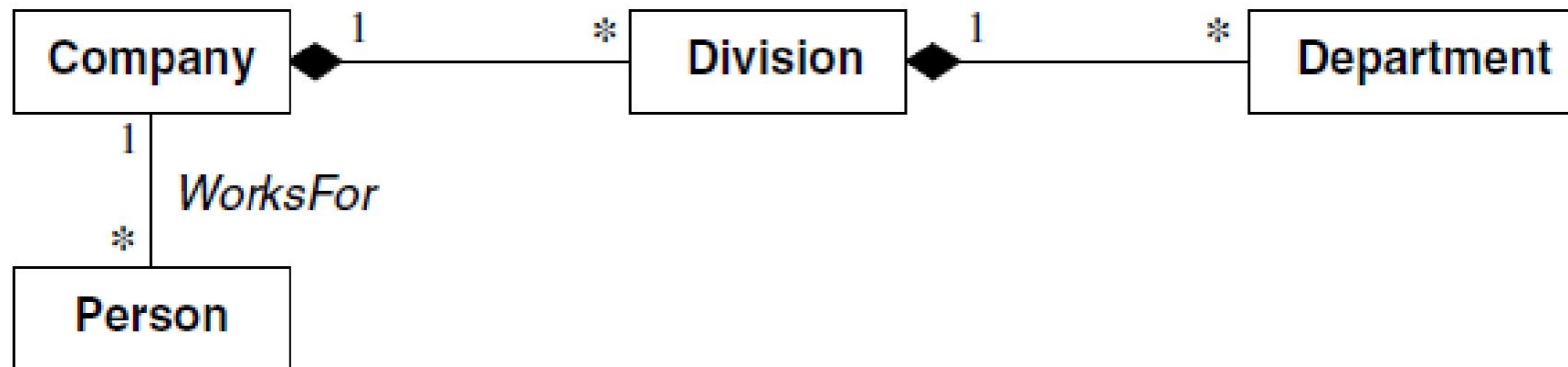
The UML has two forms of part-whole relationships: a general form called *aggregation* and a more restrictive form called *composition*

Composition is a form of aggregation with two additional constraints

1) A constituent part can belong to at most one assembly

Furthermore, once a constituent part has been assigned an assembly, it has a coincident lifetime with the assembly. Thus composition implies ownership of the parts by the whole

2) Deletion of an assembly object triggers deletion of all constituent objects via composition



Abstract Classes

An *abstract class* is a class that has no direct instances but whose descendant classes have direct instances

A *concrete class* is a class that is instantiable; that is, it can have direct instances. A concrete class may have abstract subclasses (but they, in turn, must have concrete descendants)

the following examples are concrete classes, not abstract



Figure 4.12 Concrete classes. A concrete class is instantiable; that is, it can have direct instances.

Constraint

S
A **constraint** is a boolean condition involving model elements, such as objects, classes, attributes, links, associations, and generalization sets. A constraint restricts the values that elements can assume. You can express constraints with natural language or a formal language such as the Object Constraint Language (OCL)

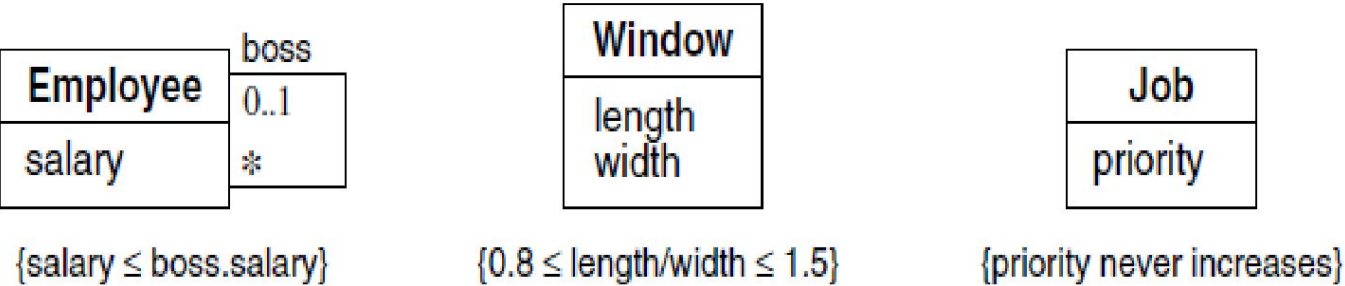


Figure 4.23 Constraints on objects. The structure of a model expresses many constraints, but sometimes it is helpful to add explicit constraints.

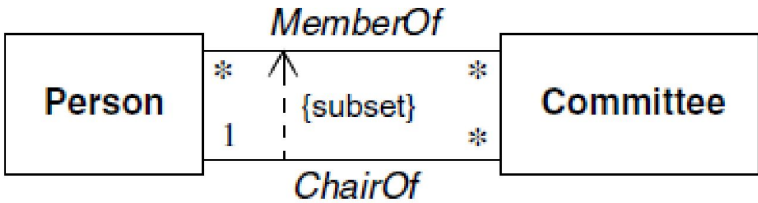


Figure 4.24 Subset constraint between associations.

Packages

A *package* is a group of elements (classes, associations, generalizations, and lesser packages) with a common theme.

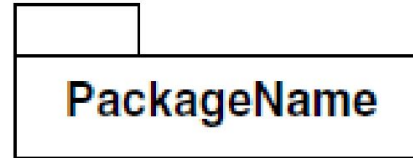


Figure 4.27 Notation for a package. Packages let you organize large models so that persons can more readily understand them.

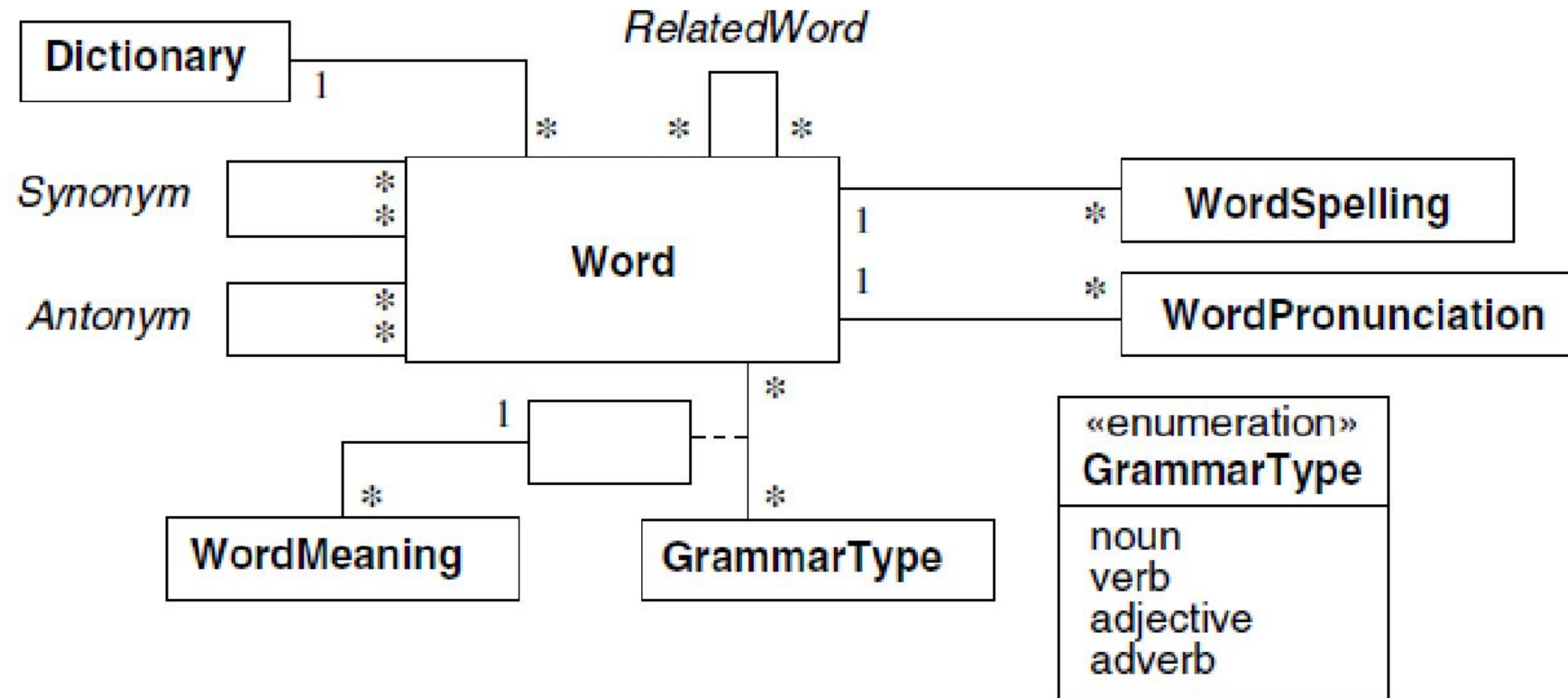
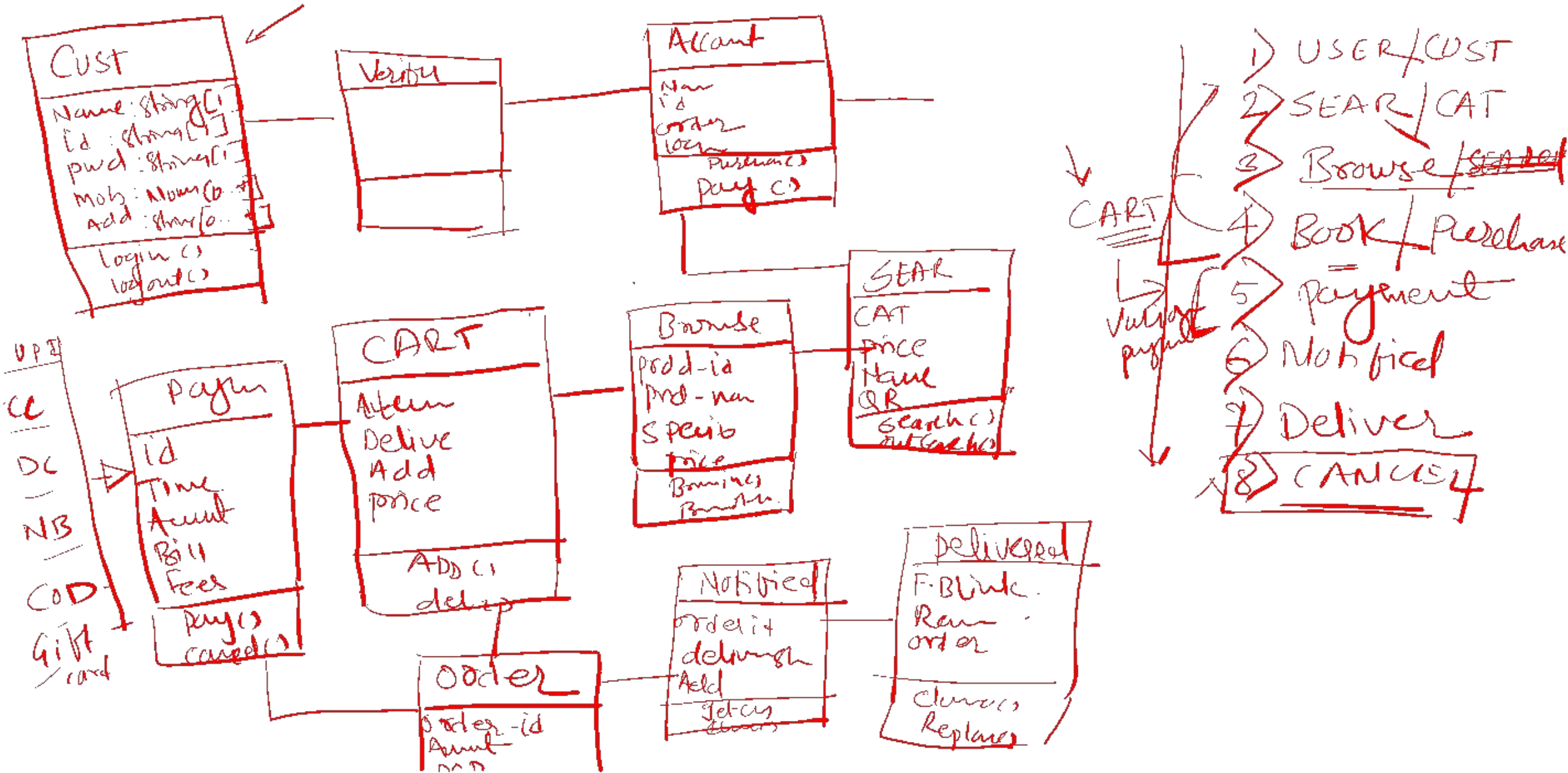


Figure E4.8 Partial model for words in a dictionary

Draw a class diagram for Purchasing a product from E-commerce Website like Flipkart/Amazon



State

Modeling

State machine diagram is a behavior diagram which shows discrete behavior of a part of designed system through finite state transitions. State machine diagrams can also be used to express the usage protocol of part of a system. Two kinds of state machines defined in **UML 2.4** are

- behavioral state machine, and
- protocol state machine.

Event

An **event** is an occurrence at a point in time, such as *user depresses left button* or *flight 123 departs from Chicago*

There are several kinds of events. The most common are the signal event, the change event, and the time event

- Signal
- Event
- Change
- Event
- Event
- Event

Signal Event

A **signal** is an explicit one-way transmission of information from one object to another. It is different from a subroutine call that returns a value. A **signal event** is the event of sending or receiving a signal. Usually we are more concerned about the receipt of a signal, because it causes effects in the receiving object.

Note

the difference between *signal* and *signal event*—a signal is a message between objects while a signal eve

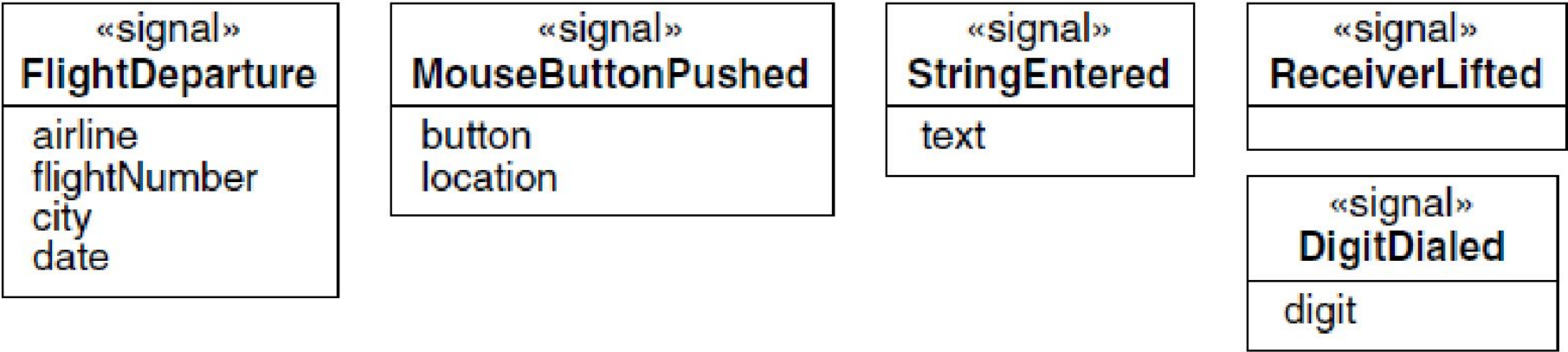


Figure 5.1 Signal classes and attributes. A signal is an explicit one-way transmission of information from one object to another.

Change

Event

A *change event* is an event that is caused by the satisfaction of a boolean expression. The intent of a change event is that the expression is continually tested—whenever the expression changes from false to true, the event happens

The UML notation for a change event is the keyword *when* followed by a parenthesized boolean expression

- when (room temperature < heating set point)
- when (room temperature > cooling set point)
- when (battery power < lower limit)
- when (tire pressure < minimum pressure)

Figure 5.2 Change events. A change event is an event that is caused by the satisfaction of a boolean expression.

Time

Event

A *time event* is an event caused by the occurrence of an absolute time or the elapse of a time interval.

the UML notation for an absolute time is the keyword *when* followed by a parenthesized expression involving time. The notation for a time interval is the keyword *after* followed by a parenthesized expression that evaluates to a time duration.

- *when* (date = January 1, 2000)
- *after* (10 seconds)

Figure 5.3 Time events. A time event is an event caused by the occurrence of an absolute time or the elapse of a time interval.

State

A **state** is an abstraction of the values and links of an object. Sets of values and links are grouped together into a state according to the gross behavior of objects.



Figure 5.4 States. A state is an abstraction of the values and links of an object.

A state specifies the response of an object to input events

State: *AlarmRinging*

Description: alarm on watch is ringing to indicate target time

Event sequence that produces the state:

setAlarm (targetTime)

any sequence not including *clearAlarm*

when (*currentTime* = *targetTime*)

Condition that characterizes the state:

alarm = on, alarm set to *targetTime*, $targetTime \leq currentTime \leq targetTime + 20$ seconds, and no button has been pushed since *targetTime*

Events accepted in the state:

event	response	next state
when (<i>currentTime</i> = <i>targetTime</i> + 20)	<i>resetAlarm</i>	<i>normal</i>
<i>buttonPushed</i> (any button)	<i>resetAlarm</i>	<i>normal</i>

Transitions and Conditions

A **transition** is an instantaneous change from one state to another.

Example:- when a called phone is answered, the phone line transitions from the *Ringing* state to the *Connected* state.

Note:- The transition is said to **fire** upon the change from the source state to the target state

A **guard condition** is a boolean expression that must be true in order for a transition to occur.

Example:- a traffic light at an intersection may change only if a road has cars waiting.

Note:- A guarded transition fires when its event occurs, but only if the guard condition is true

State Diagrams

A *state diagram* is a graph whose nodes are states and whose directed arcs are transitions between states.

Note

- 1) A state diagram specifies the state sequences caused by event sequences.
- 2) State names must be unique within the scope of a state diagram

The *state model* consists of multiple state diagrams, one state diagram for each class with important temporal behavior

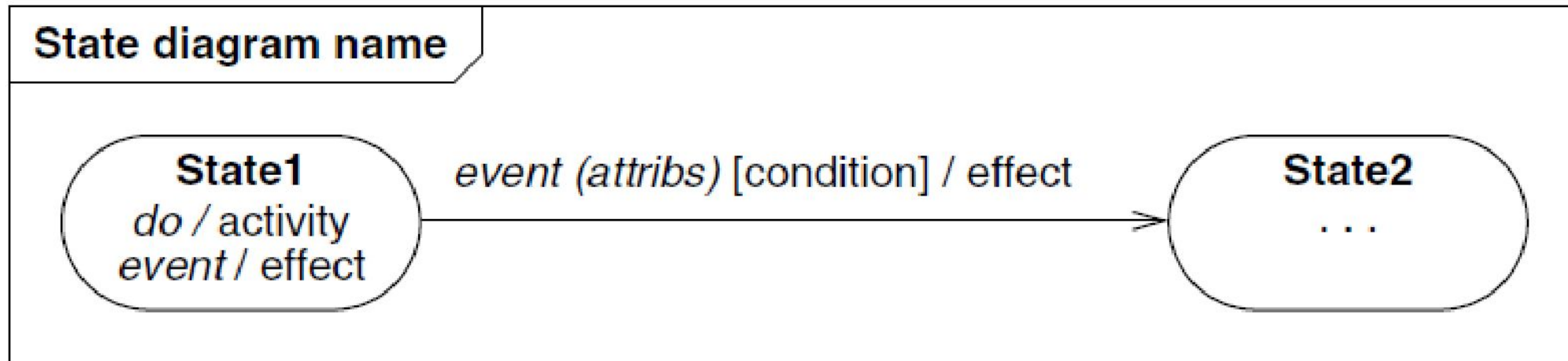


Figure 5.11 Summary of basic notation for state diagrams.

Draw a state diagram for train headlight

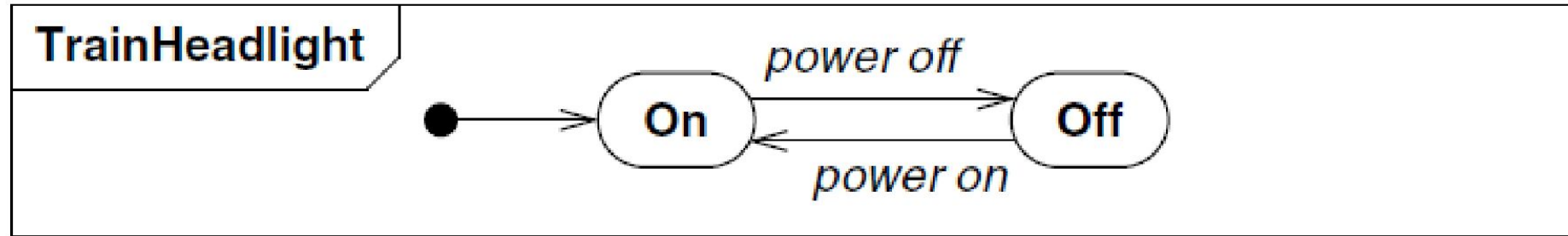


Figure A6.1 State diagram for a toy train headlight

Draw a state diagram for Train Wheels of a toy train

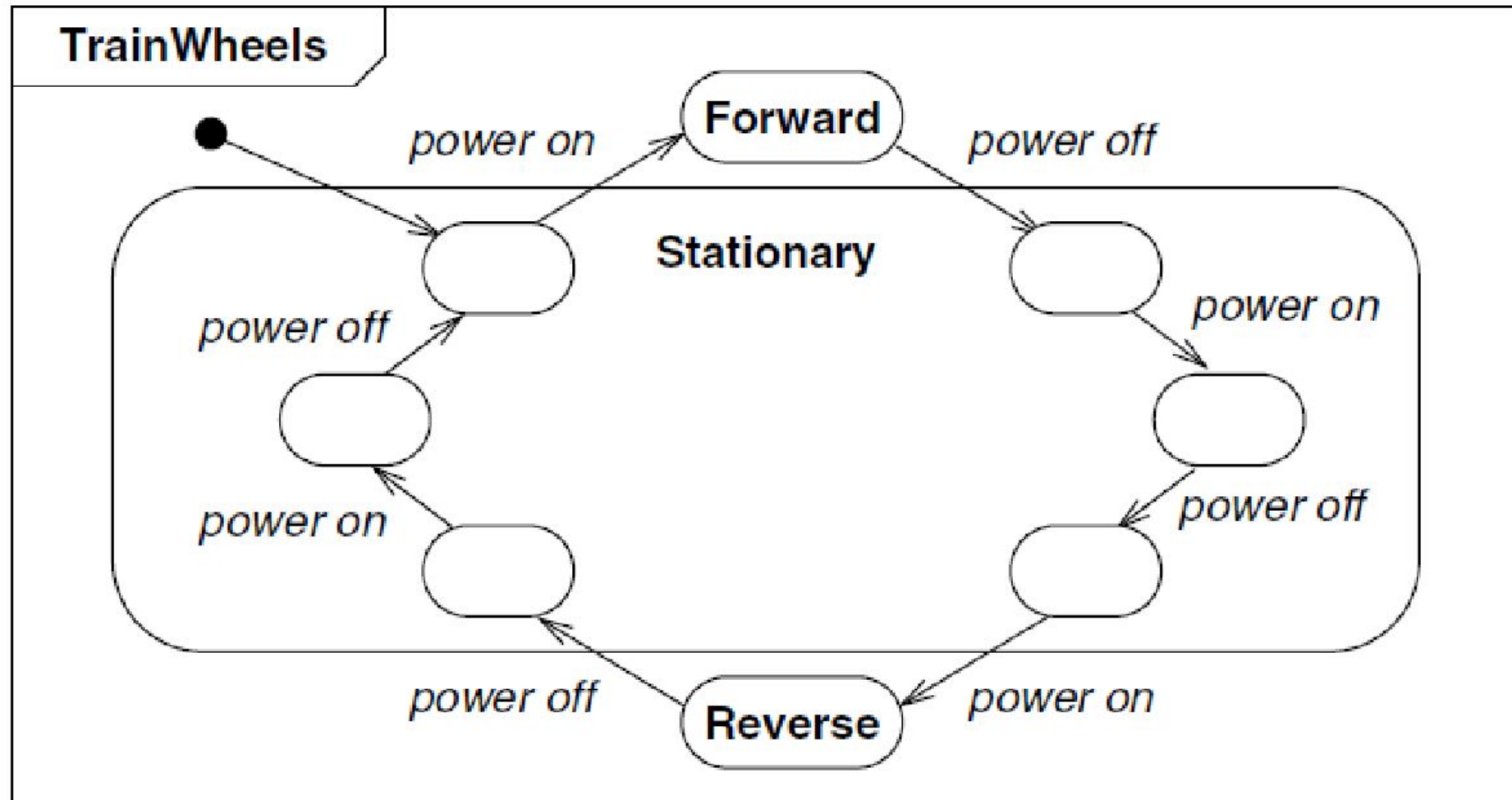


Figure A6.2 State diagram for the wheels of a toy train

- 5.2 (4) A simple digital watch has a display and two buttons to set it, the A button and the B button. The watch has two modes of operation, display time and set time. In the display time mode, the watch displays hours and minutes, separated by a flashing colon.
- The set time mode has two submodes, set hours and set minutes. The A button selects modes. Each time it is pressed, the mode advances in the sequence: display, set hours, set minutes, display, etc. Within the submodes, the B button advances the hours or minutes once each time it is pressed. Buttons must be released before they can generate another event. Prepare a state diagram of the watch.

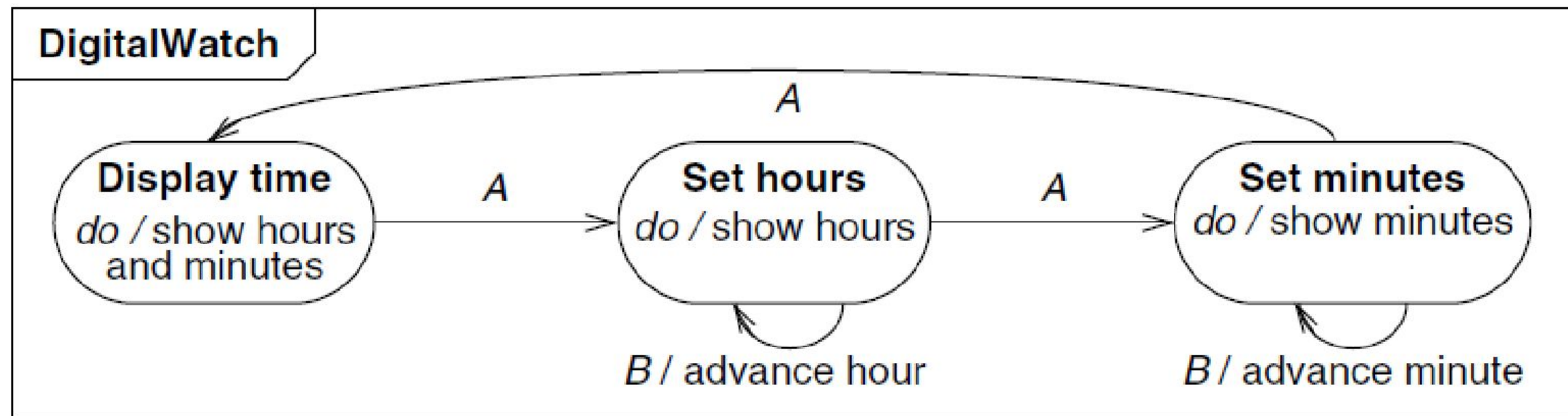


Figure A5.2 State diagram for a simple digital watch

The machine detects an incoming call on the first ring and answers the call with a prerecorded announcement. When the announcement is complete, the machine records the caller's message. When the caller hangs up, the machine hangs up and shuts off. Place the following in the diagram: call detected, answer call, play announcement, record message, caller hangs up, announcement complete.

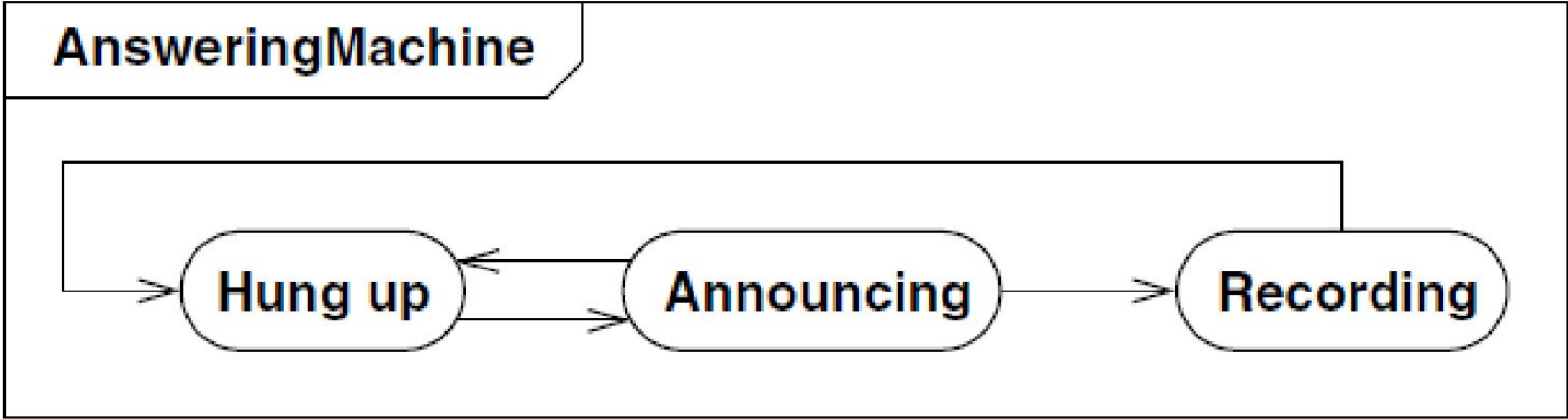
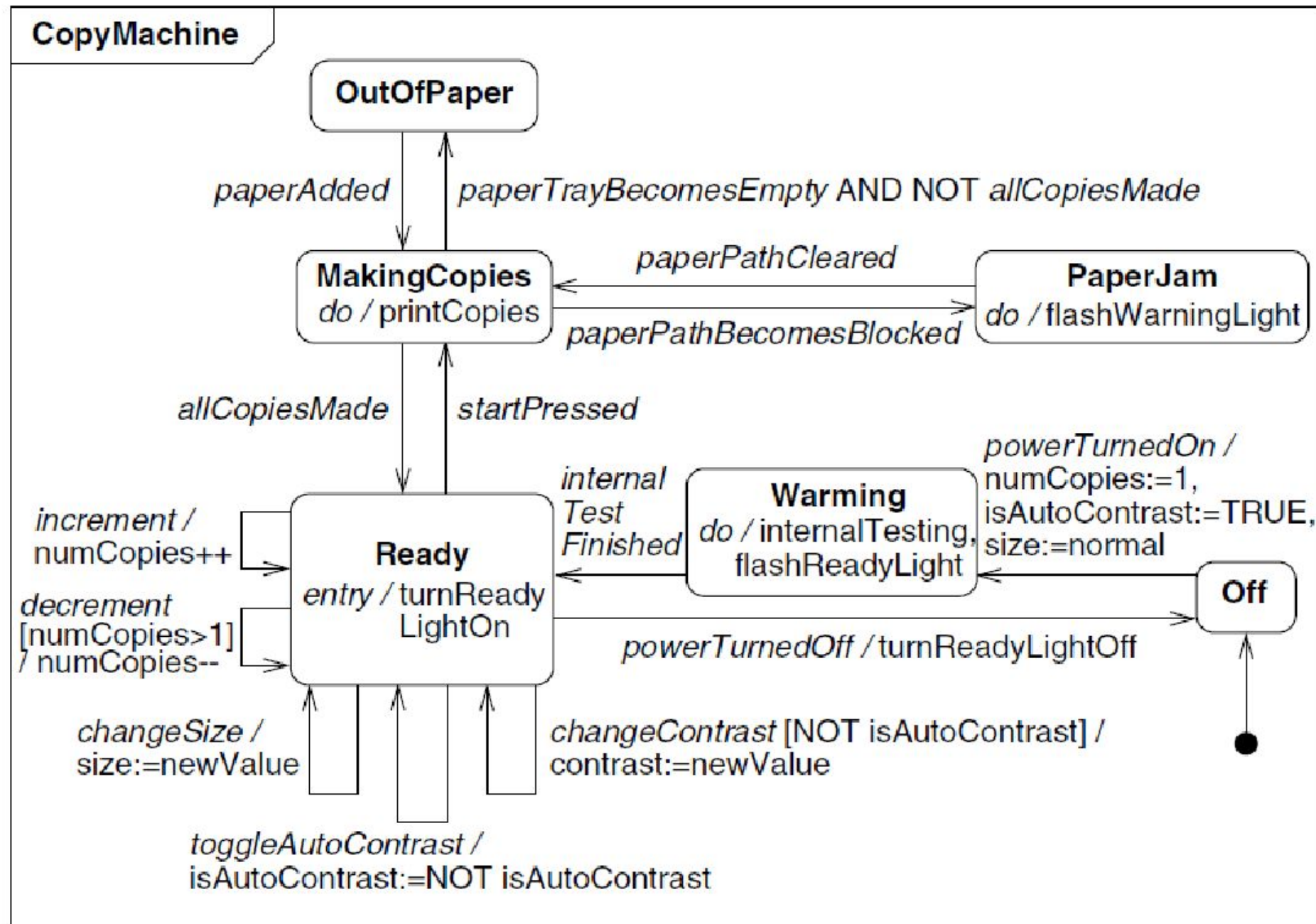


Figure E5.1 Partial state diagram for an answering machine

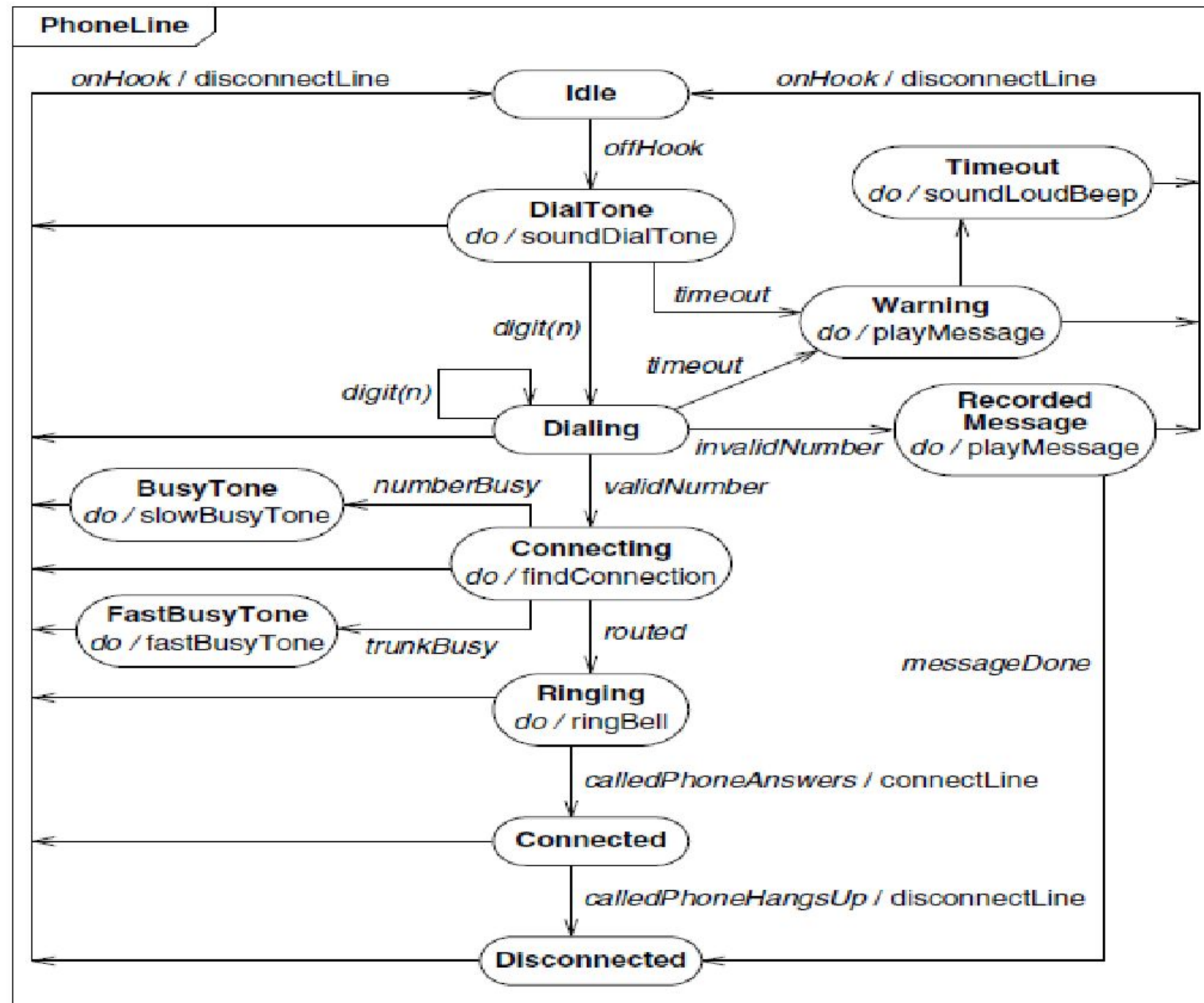
Initially the
and normal :
light stops fl
the machine
and manual
pushes the s
may jam or
making copi



contrast,
, the ready
eters when
automatic
operator
e machine
me

Figure E5.4 State diagram for a copy machine

Prepa



State diagram for chess game

